Cursor AI란 무엇인가?

- AI 기반 코드 에디터
- VS Code 기반 확장형 개발 환경
- 프로젝트 전체 문맥(Context)을 이해하는 LLM 통합

단순 자동완성 도구가 아니라 대화형 코드 파트너

Cursor AI란 무엇인가? (VS Code의 진화)

- **정의:** VS Code 기반의 AI 전용 코드 에디터 (기존 확장 프로그램/설정 100% 호환)
- **핵심 철학: 프로젝트 전체 코드(Context)를 이해하는 에디터**
- **차별점:**
  - GitHub Copilot 대비 깊은 수준의 코드베이스**(Codebase)** 이해도
  - 최신 LLM(Claude 3.5 Sonnet, GPT-4o) 실시간 교체 사용 가능

Cursor가 하는 일

코드 생성 및 설명

버그 분석 및 리팩토링 제안

테스트 코드 생성

문서화 자동 생성

AI가 작성한 코드는 무조건적 '검증 대상'

설치 및 환경 설정 (Setup Guide)

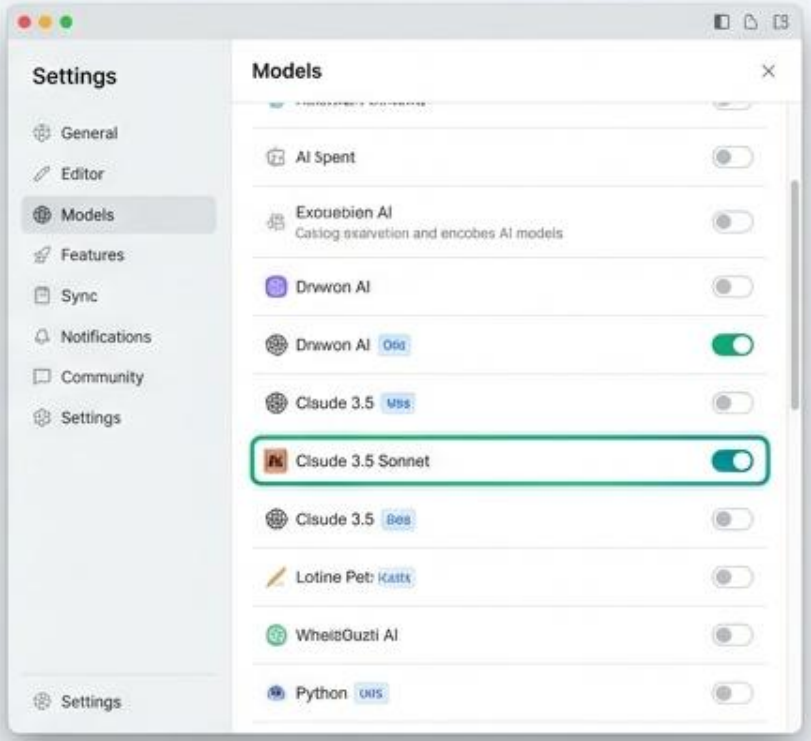
1 설치 → 2 계정 연동 → 3 환경 설정 → 4 시작하기

1. **설치:** cursor.com 접속 및 OS 버전 다운로드

2. **계정 연동:** GitHub 또는 Google 계정 로그인

3. **환경 설정 (중요):**
   - Import from VS Code 선택 (기존 테마, 단축키 유지)
   - Settings → Models: Claude 3.5 Sonnet 활성화 권장

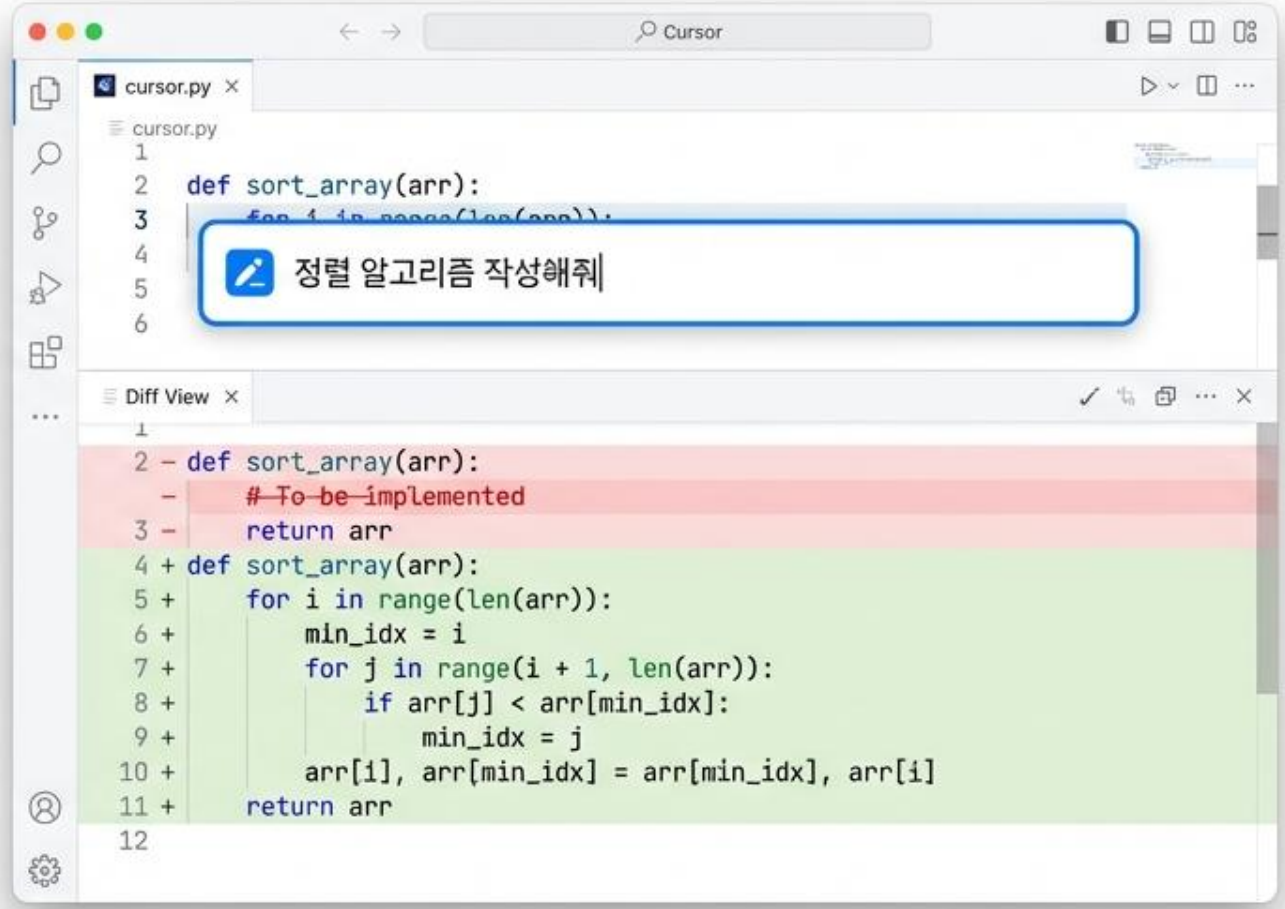
4. **시작하기:** File → Open Folder로 실습용 C/Python 프로젝트 폴더 열기



핵심 기능 1: Inline Edit (즉석 코드 생성)

⌨ Cmd(Ctrl) + K

- **용도**: 코드 창에서 즉석으로 코드 생성 및 수정 요청
- **사용법**:
  1. 수정할 코드 드래그(선택) 또는 빈 줄 클릭
  2. 단축키 입력 후 프롬프트 작성
- **학습 포인트**: Diff(차이점) 보기 모드 활용 → **AI의 수정 내역 비교 분석 필수**



코드 내에서 바로 수정: Inline Edit

이 코드를 더 Pythonic하게 리팩토링해줘.

```
def calculate_area(length, width):
    area = length * width
    return area

# This is a simple function
```

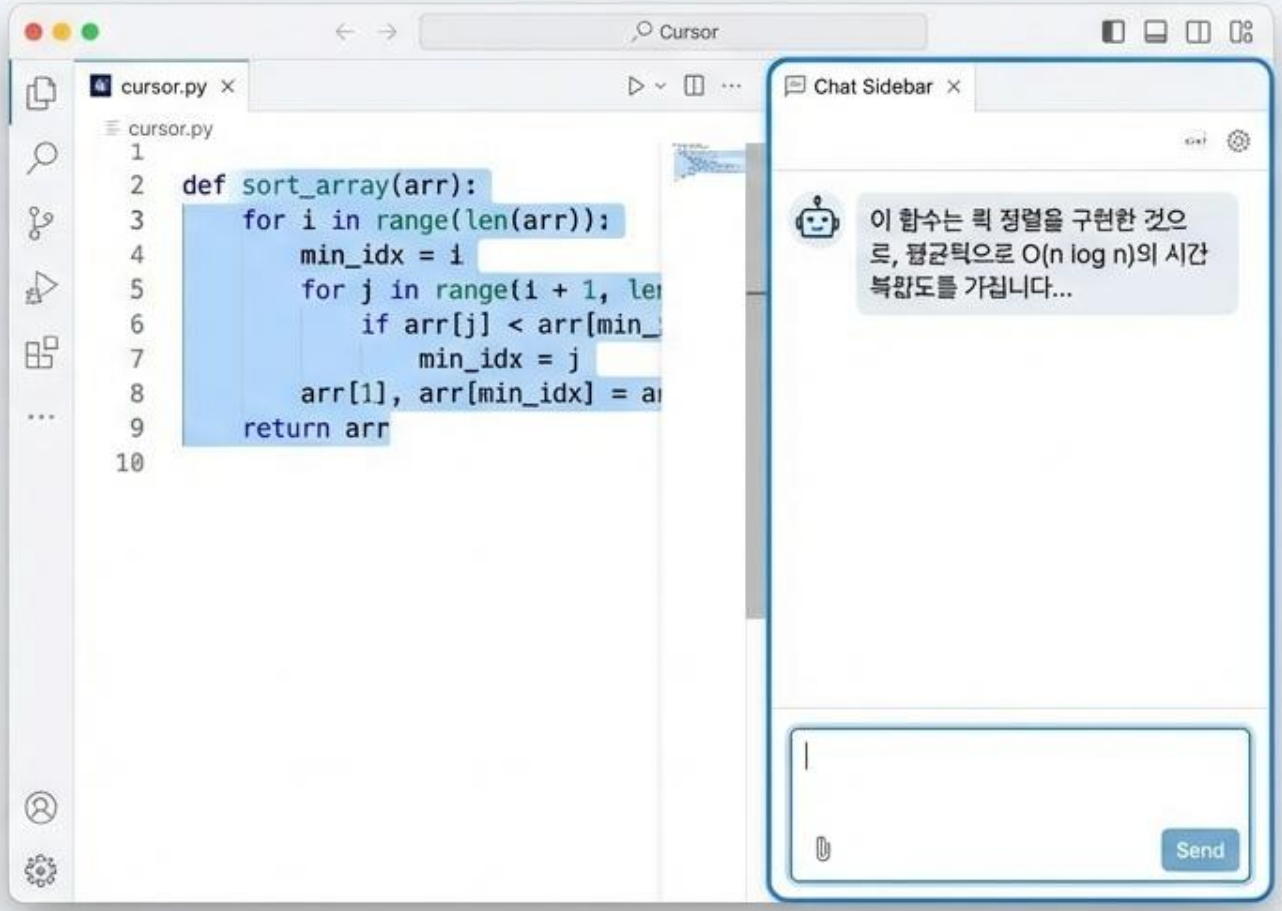
Ctrl + K

AI가 에디터 내에서
코드를 즉시 변경

핵심 기능 2: AI Chat (코드 대화 및 튜터링)

⌨ Cmd(Ctrl) + L

- **용도**: 코드의 원리 파악 및 질문 (1:1 튜터 역할)
- **사용법**:
  1. 설명이 필요한 코드 영역 선택(Highlight)
  2. 단축키 입력 후 질문 작성
- **질문 예시**:
  - 이 Python 함수의 시간 복잡도가 왜 O(n log n)이야?
  - 이 부분을 더 간결하게 리팩토링해줘.



가장 기본적인 사용법: Chat

```python
def calculate_area(length, width):
    area = length * width
    return area

# This is a simple function
```

이 함수에 타입 힌트를 추가하고
예외 처리를 넣어줘.

기존 코드를 읽고 분석...
수정된 코드 제안.

핵심 기능 3: @ Symbols (Context Targeting)

AI에게 참고할 정보를 명시적으로 지정하는 기능

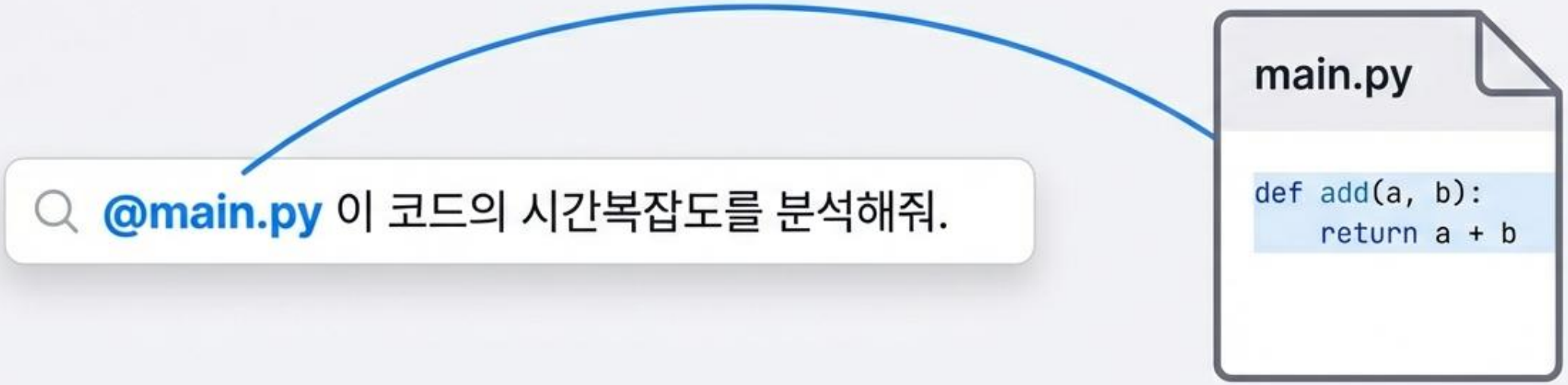
@Files
(특정 파일 내용 참고)

@Codebase
(프로젝트 전체 구조 및 연관성 파악)

@Web
(최신 라이브러리/기술 인터넷 검색)

@Docs
(공식 문서 기준 답변 요청)

파일 참조 기반 질문 (Contextual Query)



- **@ 기호의 역할:** 특정 파일이나 문서를 AI에게 '참조'로 전달
- **Cursor의 처리:** 해당 파일의 내용을 문맥(Context)에 포함하여 답변 생성

C 언어 코드 분석 예제

질문: 이 코드의 잠재적 오류를 찾아줘.

```c
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d", a / b);
}
```

AI 분석 결과: 0으로 나누기 문제 (Division by Zero)

[실습 1] Python: 예외 처리와 데이터 안전성

Mission: 예외 처리가 포함된 견고한 함수 작성

1. **Generate (Ctrl+K)**: '리스트의 평균을 구하는 함수 작성해줘'
2. **Refine (Ctrl+L)**: '입력값이 비어있을 때 발생하는 에러(ZeroDivisionError) 방지 코드 추가해줘'
3. **Analyze**: try-except 구문 확인 및 주석 작성

```python
def calculate_average(numbers):
    try:
        if not numbers:
            return 0
        return sum(numbers) / len(numbers)
    except TypeError:
        return None  # 데이터 타입 에러 처리
```

[실습 2] C 언어: 메모리 관리와 구조체

Mission: 동적 할당의 이해 및 메모리 누수 방지

1. **Generate (Ctrl+K): "학생 정보를 담는 구조체와 동적 할당 코드 작성해줘"
2. **Verify (Ctrl+L)*: "이 코드에서 메모리 누수(Leak)가 발생할 가능성이 있는 부분 찾아줘"
3. **Deep Dive**: malloc과 free의 짝 확인

```
typedef struct {
  char name[50];
  int age;
} Student;

// ... memory allocation code
...
free(student_ptr); // 메모리 해제 필수!
```

Memory Cleanup

결론: AI 시대를 대하는 개발자의 자세

도구와 책임

AI는 도구일 뿐, 결과물에 대한 책임은 온전히 개발자에게 있음

능동적 학습 (Active Learning)

코드를 눈으로만 보지 말고 직접 타이핑하며 체득하기

질문의 힘

질문이 구체적이고 논리적일수록 더 좋은 코드가 나온다